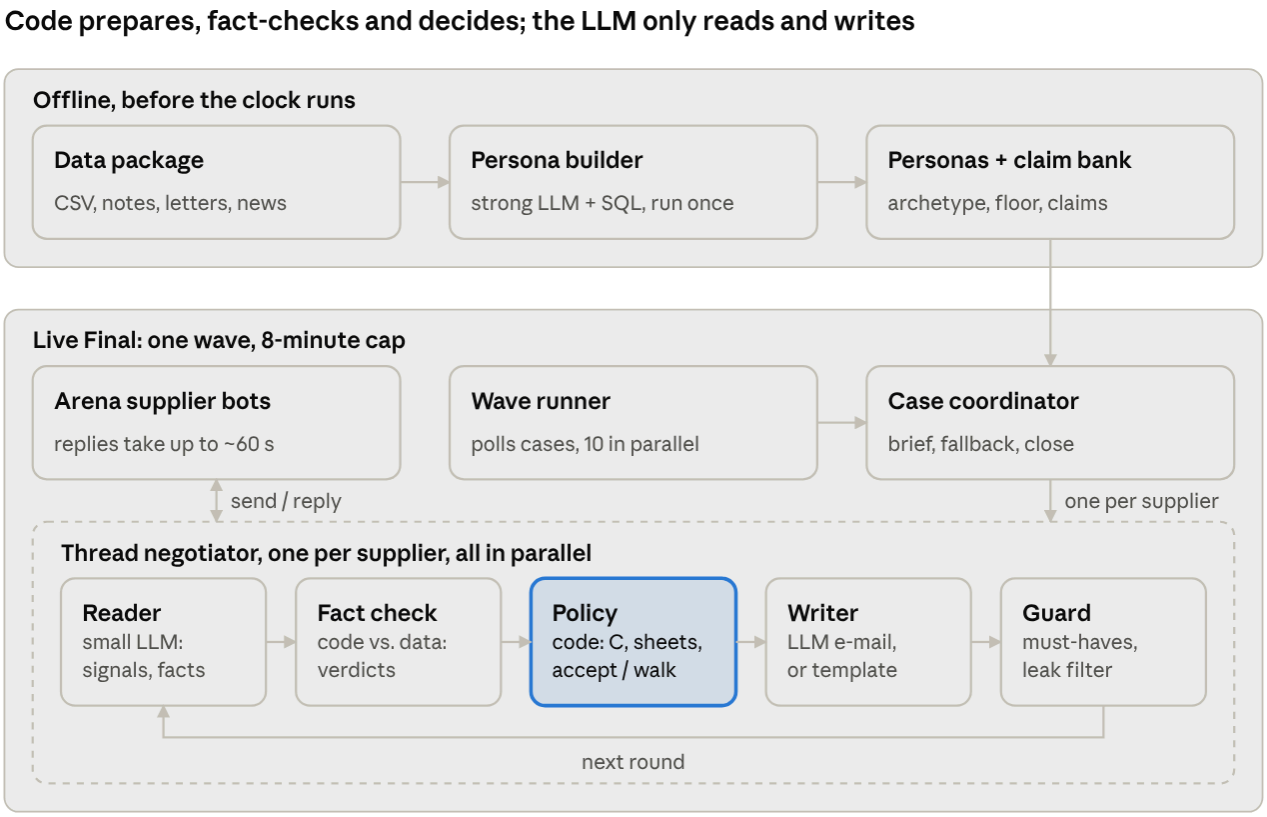
Buyer Agent: How the System Works

Oct 10, 2026 · @Julian

Our agent negotiates each case as a team of parallel thread negotiators under one case coordinator: code prepares the case from data, checks every supplier fact, and sets every term; the LLM only reads replies and writes e-mails.

Architecture



system architecture · offline preparation, live wave, thread negotiator loop

Personas and claims are built once before the final. During a wave, every supplier thread loops reader → fact check → policy → writer → guard; only the code steps set numbers.

A case, step by step

1. **Wave starts.** The wave runner sees the new cases and starts one case coordinator per case, up to 10 in parallel.
2. **Prepare (within 20 s).** The coordinator opens the session and builds a case brief from data: costs (C_open, our alternative, target, walk-away), the lever table, a view of each supplier, market facts and a claim bank of verified arguments. Unknown suppliers or

parts go through the fallback cascade. A stance call proposes each supplier's negotiation parameters, clamped by code.

3. **Negotiate in parallel.** One thread negotiator per supplier runs a loop each round:
   a. The reader turns the supplier's e-mail into signals and factual statements.
   b. The fact checker tests each statement against our data.
   c. The policy engine picks the next term sheets and claims, or decides to accept or walk.
   d. The writer turns that decision into an e-mail.
   e. The guard checks it and sends it.
4. **Close.** The coordinator accepts the best sheet that keeps every must-have and beats our alternative, or walks away. It closes at the latest 90 s before the wave's cap.
5. **Learn.** The full trace and what we learned about each supplier are stored for the next practice run.

The modules

Module	Job	In → out	Run by
Wave runner (`runner.py`)	Polls the final's case list, starts a coordinator per case, restarts after a crash	Case list → running cases	Code
Case coordinator (`agent.py`)	Builds the case brief, starts thread negotiators, keeps the shared blackboard, decides when to close	Case file + personas + data → brief, final accept or walk	Code
Fallback cascade	Fills gaps for unknown suppliers or parts: case facts, database search, category averages, the supplier's own documents, web context last	Unknown ID → filled fields with source and confidence	Code; web search time-boxed
Personas (`personas.py`, `knowledge/`)	One profile per supplier: archetype with evidence, levers, tone, history	Data + past negotiations → persona JSON	LLM offline, code live

Module	Job	In → out	Run by
Claim bank (`claims.py`)	Arguments as sentences with figure and data ID, computed exactly as the arena checks them	Data → verified claims, ranked per supplier	Code
Economics (`economics.py`)	Total cost C of any sheet, floor, anchor, target, probe sheets, concession steps, accept gate	Case + brief + stance + signals → term sheets and limits	Code
Reader (`llm.py`)	Reads a supplier e-mail: signals (approval needed, walk risk, mood, cheaper variant) and factual statements	E-mail → JSON	Small LLM, hard timeout
Fact checker	Tests each supplier statement against the data: confirmed, contradicted, selective or unverifiable (not built yet)	Statements → verdicts and rebuttal claims	Code
Policy engine	Chooses the next move from goal, case type, archetype, verdicts and time left	Brief + thread state → sheets, claims, tone, or accept/walk	Code
Writer (`llm.py`)	Turns the chosen move into a short business e-mail	Move → e-mail text	Small LLM; template if late
Guard (`guard.py`)	Blocks leaks, unknown IDs, figures the plan did not set, broken must-haves; adds missing claims	Draft → safe message	Code
Knowledge and logs	Append-only negotiation log, full traces, per-move log, findings	Every move and trace → files in git	Code

Most important data

Nine data sources feed the case brief, ranked by impact on the score: the case file and the quote decide C directly; the rest decide how far a supplier will move and why.

#	Data	What it decides
1	Case file and quote PDF: need, horizon, rates, must-haves, alternative, term options, standard terms	C directly, including the uncovered-need trap
2	Supplier cost floor: should-cost, cost drivers, cost-breakdown spreadsheet	Anchor, target and the margin penalty
3	Index values for the supplier's cost drivers	The justified price change, our strongest verified claims, the basis of the fact check
4	Letters, notes and news per supplier	What actually moves it (cash, capacity, relationship), plus notice periods
5	Negotiation history: asked vs. settled per supplier	The best prior for how far it moves
6	Supplier intel and our invoice history: credit index, equity, ownership, our late payments	The archetype and the payment lever
7	Contracts, clauses and sourcing	Notice periods, index clauses, expiry dates, single-sourcing
8	Scorecards and quality notifications	The quality lever
9	Forecasts, past order prices, rival quotes	Volume growth, price history and rivals as arguments

Stance and fallback constants

Since draft two, the opening gap, capture, margin, justified share, concession steps and the margin estimated in the quote form a **stance** that the model proposes per supplier at case open, from the dossier (profile, signals, letter, quote, cost data, rivals). Code clamps each value to a range by tier and goal and logs it with a one-sentence reason. The constants below are the fallback when the stance call times out.

Tiers come from the persona: strategic covers castings, seals and electrical parts, A suppliers and single sources; important and standard follow the persona's tier.

Fallback constant	Values	What it means	Example
`GOAL_CAPTURE`	leverage 0.75, routine 0.5, stabilise 0.6, secure_supply 0.55, quality_first 0.65	Share of the possible saving (opening quote's C down to our best estimate) to reach before accepting; higher means pushing longer	Leverage case: accept once 75 % of the possible saving is in hand
`OPENING_GAP`	strategic 0.05, important 0.08, standard 0.12	How far below the supplier's quote the first offer anchors; never below the floor; not used in price-increase cases	Quote €100, standard supplier: first offer €88
`TIER_MARGIN`	strategic 4 %, important 2 %, standard 0 %	Margin left above the supplier's estimated full cost when setting the floor. In `stabilise` / `secure_supply` the case's `sustainable_margin_pct` replaces it; standard suppliers are capped at 3 %	Full cost €90, strategic: floor €93.60
`JUSTIFIED_SHARE`	strategic 0.5, important 0.25, standard 0.0	In price-increase cases, share of the data-justified increase offered in round 1	Justified +2.3 %: strategic first offer +1.2 %, standard +0 %
`STEPS`	35 %, 25 %, 15 %, 10 % of the remaining gap; 60 % once the tier's price pushes are used	Size of each concession, made only after the supplier moved; the 60 % step closes fast when we may not push further	Gap €10: concede €3.50, then €1.63, then €0.73

How a stance becomes a live value (clamps in code, every value logged with its reason):

1. **Stance:** the model's proposal for this supplier, or the fallback constant for its tier and goal.
2. **Clamps and data checks:** each value stays inside its tier and goal range. The anchor never goes below the floor, built from full cost in order of trust: the supplier's cost-breakdown XLSX, else should-cost, else the quote less the stance's margin estimate. Price increases start from the weighted justified increase.

3. **Live signals, within fixed bounds:** `needs_approval` multiplies steps by 0.7; `walk_risk: high` stops price pushes; a cool mood warms the next e-mail but keeps the terms.

Tuning rule: one constant per batch of 10 practice cases, each case replayed twice before a change is credited, every result logged in `docs/findings.md`.

Draft-two results (sandbox, 10 Oct)

On the free sandbox, draft two reached 0.96 on D01 and 0.54 on D02, with every claim verified after run 1 and no timeouts. No practice quota was used. Latency was 11-19 s per move (reader plus writer, `claude-opus-5-5` at low effort); the stance call adds 9-11 s once at case open.

Run	D01: price increase, strategic foundry	D02: RFQ, two job shops	What changed
1	0.73, deal €50.90, 2 rounds	0.37, deal €15.30, 3 rounds	Draft-two port, constants from the plan
2	0.93, deal €49.90, 4 rounds	0.43, deal €15.00	Own-world claims, supplier-friendly tie-breaks, floor below the opening gap, hold a round while the supplier moves
3	0.84, agreed €50.37, 2 rounds	0.32, deal €15.50	First model-proposed stance (too cautious)
4	0.84 (bound by the same floor)	0.54, final offer €14.50, 5 rounds	Calibrated stance prompt, 3 % margin cap for standard suppliers, closer waits while the supplier moves
5	0.96, agreed €49.65, 4 rounds	not run	Floor on the case's sustainable margin

Replays: D01 · D02. The single contradicted claim (run 1) was a package ID cited in the demo world.

What changed, and why

- **A real floor every time.** Before, a supplier without should-cost got a floor at 90 % of its quote, which clipped the opening gap.

- **The margin we leave is the case's number.** The arena charges only below `sustainable_margin_pct`, and only in `stabilise` and `secure_supply`. Draft one's 8 % strategic rule bound the D01 anchor at 0.84; the case's 4 % gave 0.96.
- **The stance needed calibration.** Its first proposals were more cautious than the constants (7 % gap, margin on top of margin) and D02 fell to 0.32. A paragraph with the concessions seen so far plus the 3 % cap fixed it: 0.54.
- **Closing.** At the target, hold one more round while the supplier still moves and rounds remain; the stall rule no longer accepts a moving supplier. In run 1, D01 accepted after 2 rounds with 8 left.
- **Claims.** A case whose refs the package does not know cites only its own facts. Rival offers are cited only after the bidder's first reply and only when cheaper.
- **Sheets.** Among options that cost us the same, take the one the supplier prefers: 35 days' lead time instead of 21, which the job shop had priced with overtime.

Next

- [ ] Price-increase anchor for strangers: check the letter's own figures (wage +3.1 % on a 28 % share) against the index series and use them as the justified figure.
- [ ] Stop asking for cheaper variants: they need engineering approval, cost a round and never become a binding sheet.
- [ ] Calibrate capture: the stance chose 0.65-0.8, yet both bots reached their final offer in 4-5 rounds regardless. Compare with where suppliers stop in practice; one hypothesis per batch.
- [ ] Build the fact check: the reader does not yet extract the supplier's assertions, so the strongest reply, a `selective` verdict naming the omitted driver, is unused.
- [ ] Keep the three probe sheets (suppliers answered the one they liked in every run); add the index clause as a probe in renewals.
- [ ] In the final, run the stance call from the case list before the session opens, and use the headroom in the 15-20 s per move for the fact-check call.

Design rules

- **Code owns every number and every binding decision.** The LLM never sets a term, a price or an accept; it reads and writes.
- **Score is total cost, not price.** Every sheet is valued with C, including payment days, effective date, coverage, freight, quality and the supplier's margin in strategic cases.
- **Threads run in parallel.** A 3-supplier RFQ gets as many rounds per supplier as a single-supplier case.

- **Every step has a fallback.** A slow model falls back to a template, an unknown supplier to the cascade, the wave cap to the closing move.
- **Every move is logged with its reason**, so any e-mail can be explained to the jury: the plan, the C of each sheet, the claims and the supplier's answer.

Detailed policies, timing budget and build plan: Negotiation Arena: Agent Policies & Orchestration.